Main idea for the tested selector:

Use the reference solver to compute the “score” for each pair of frequencies by:

```
target_list = []
for i in range(n_train):
    conductivity = compute_LAMBDA_MAT(train_x[i, 0, :, :])
    gradient, _, _ = solve_thermal_fft(conductivity, wavevectors, grad_T_MAC, lambda_0, tol=fft_tolerance, max_iter=fft_max_steps)

    fluctuation = gradient - grad_T_MAC[:, None, None]
    fluctuation_hat = jnp.fft.rfft2(fluctuation, axes=(-2, -1))

    score = jnp.sum(jnp.abs(fluctuation_hat[:, mode_x, mode_y])**2, axis=0)
    score = score * score_multiplier
    score = score[1:] # excluding the zero mode, will always present anyway
    score = score / jnp.sum(score)

    target_list.append(score)
print(f"Computed target list with shape: {jnp.asarray(target_list).shape}")
```

✓ 4.8s

```
Computed target list with shape: (256, 513)
```

1. Get the full gradient field by the reference solver
2. Compute the fluctuation field by subtracting the imposed macroscopic thermal gradient
3. FFT on that to obtain the contribution of each pair of frequencies in each direction
4. The score is then the sum of the squared modulus of the contribution in each direction: |Gx|^2 + |Gy|^2
5. Non-self-conjugate frequencies’ scores are multiplied by 2
6. Normalize the whole ‘score’ field and this field should essentially be the contribution score field for each sample

Now the neural network is trained to predict this field. Then the top ~200 pairs of frequencies are chosen to be kept. Those pairs of frequencies (in theory) are the most important pairs for a sample.

```
def select_modes(scores):
    _, indices = jax.lax.top_k(scores, k_retain - 1)
    mask = jnp.zeros(n_kept, dtype=jnp.float32)
    return mask.at[0].set(1.0).at[indices + 1].set(1.0)
```

By re-assembling the fluctuation_hat field for the test samples, using the 3 methods:

- Trained selector
- Regular, rectangle method we have been using for all the FNO models
- Oracle reference selector, automatically selects 200 pairs of frequencies with highest score for every test sample

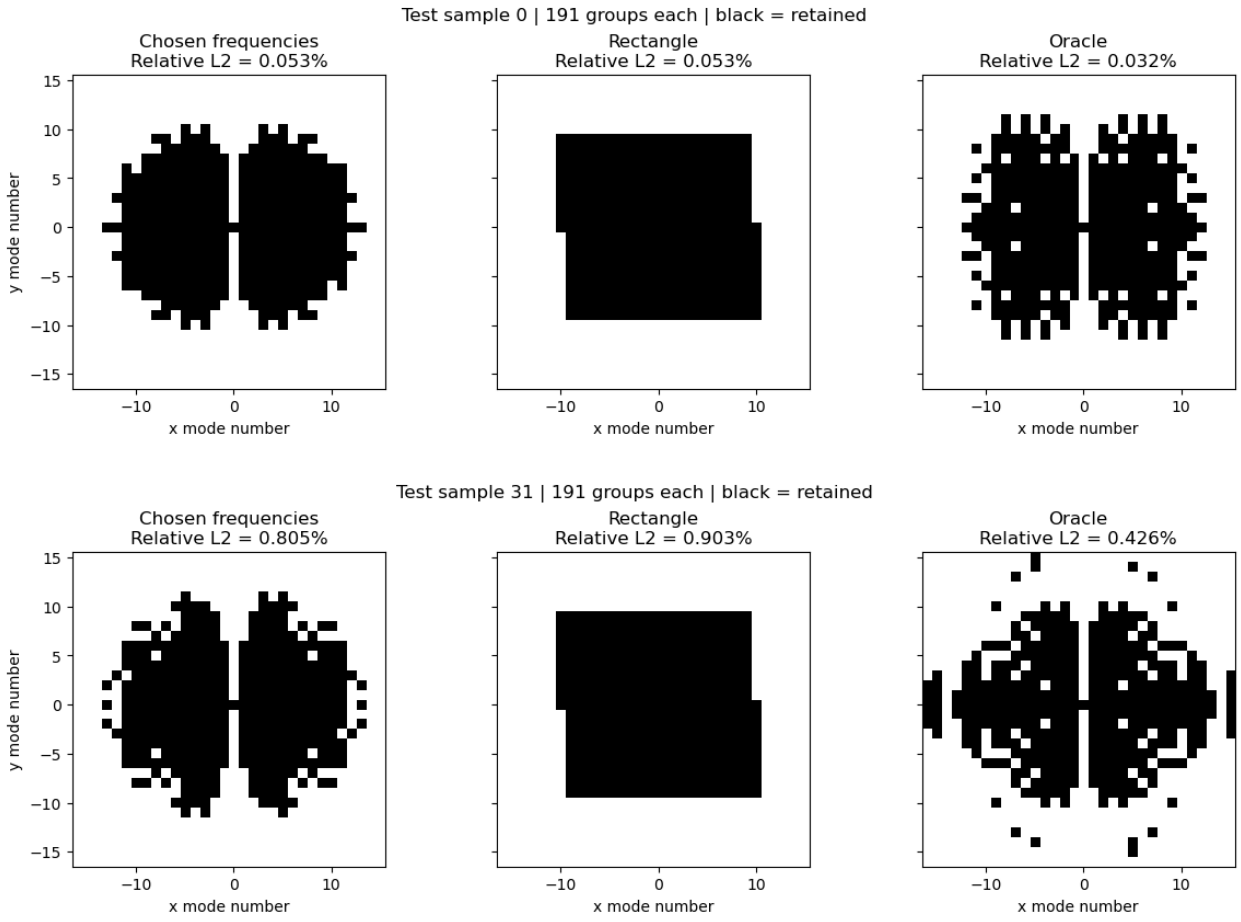
Then recreate the final full gradient field with those pairs of frequencies, then compare those 3 fields with the reference gradient field (with all frequencies used), we obtain the L2 error below:

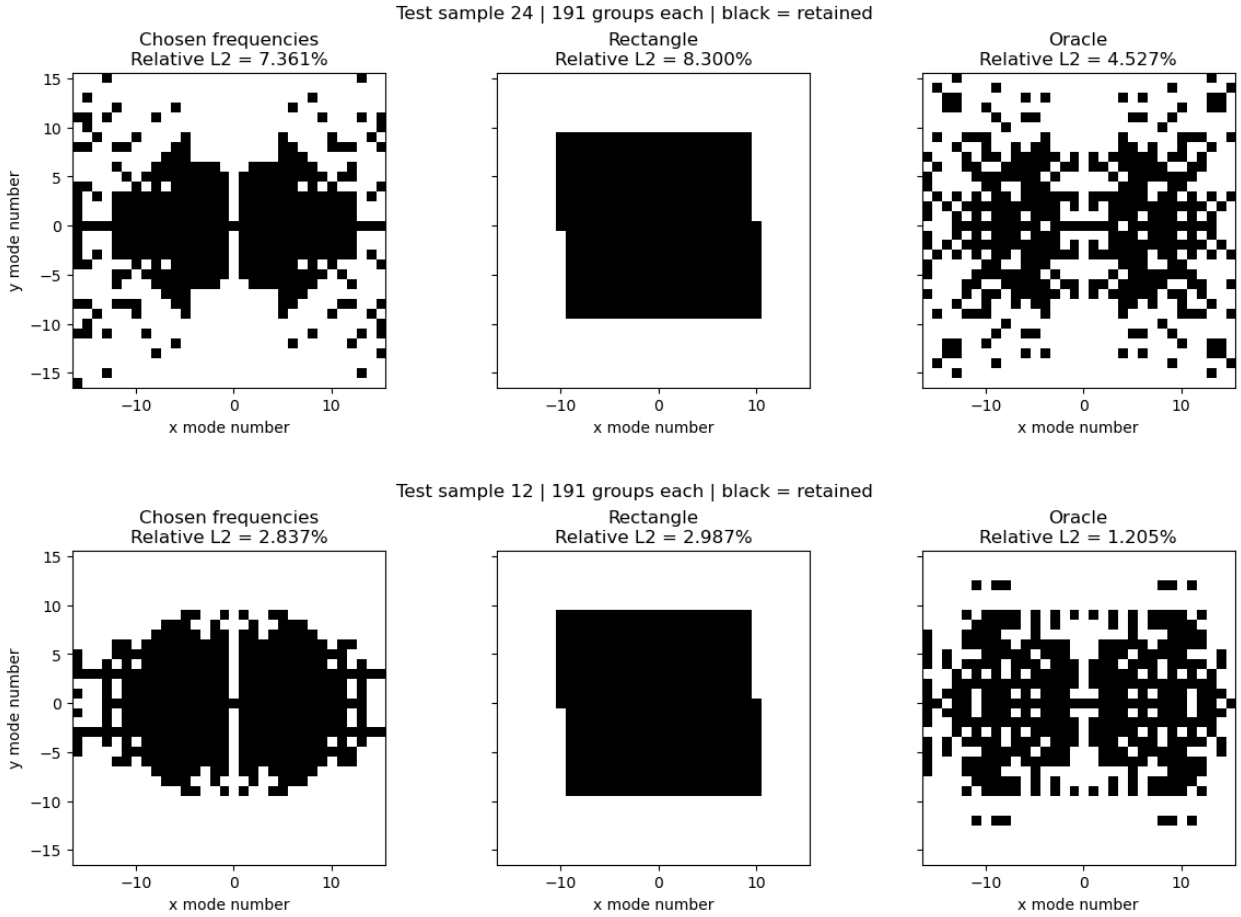
```
Mean relative L2 error — Chosen frequencies: 3.129130e-02
Mean relative L2 error — Rectangle: 3.713090e-02
Mean relative L2 error — Oracle: 1.761423e-02
```

This means that there is room for improvement.

- The trained selector successfully predicted the top 200 pairs of frequencies that makes the final gradient field more accurate (compared to the default rectangle method).
- However, it's still no where near close to the optimal 200 pairs of frequencies, so improvement could be made to improve the selector

Some examples:





Applying to FNO:

I have tried to applied the selector to the FNO, and to be honest, at least with this type of implementation, it's not really applicable in my opinion.

```
Epoch 190/200 | Loss: 1.541550e-06 | Epoch time: 613.9 s | Elapsed: 1605.4 min
Epoch 191/200 | Loss: 1.541062e-06 | Epoch time: 614.7 s | Elapsed: 1615.6 min
Epoch 192/200 | Loss: 1.538891e-06 | Epoch time: 611.9 s | Elapsed: 1625.8 min
Epoch 193/200 | Loss: 1.538730e-06 | Epoch time: 614.7 s | Elapsed: 1636.1 min
Epoch 194/200 | Loss: 1.536734e-06 | Epoch time: 617.4 s | Elapsed: 1646.3 min
Epoch 195/200 | Loss: 1.535778e-06 | Epoch time: 614.8 s | Elapsed: 1656.6 min
Epoch 196/200 | Loss: 1.535160e-06 | Epoch time: 617.8 s | Elapsed: 1666.9 min
Epoch 197/200 | Loss: 1.533386e-06 | Epoch time: 617.1 s | Elapsed: 1677.2 min
Epoch 198/200 | Loss: 1.531930e-06 | Epoch time: 625.3 s | Elapsed: 1687.6 min
Epoch 199/200 | Loss: 1.531202e-06 | Epoch time: 616.2 s | Elapsed: 1697.9 min
Epoch 200/200 | Loss: 1.531417e-06 | Epoch time: 624.5 s | Elapsed: 1708.3 min
Training complete. Continuing to notebook post-processing.
```

First of all, a glaring issue is that it doesn't improve the accuracy of the FNO. Somehow it even made it worse:

The final loss of the regular FNO with the same setting is 9.987527e-07

And with the selector, the final loss is 1.531417e-06

In my opinion, the selector implemented this way would only make sense to be used with the regular FFT solver, not with FNO since maybe the pairs of frequencies chosen by the selector are not the one most important to the neural network. It learns hidden structures and important frequencies to the FFT solver might not be the same for the FNO.

➔ We need to have the selector trained at the same time as the FNO, and have them share the same loss function

The other problem is that, every pair of frequencies available must have a separate weight, as the selector could select any pairs for a sample. This increases the number of parameters by a HUGE amount:

```
FNO parameter arrays: 12,598,562
Frozen selector parameters (outside the optimizer): 43,133,953
```

This explains why the training time is much much worse, 1708.3 mins for the FNO with selector. The regular FNO trains in 371 mins. The number of parameters explodes with a small increase in the grid size, so this implementation is not suitable for FNO at all.

➔ I think, instead of allowing the selector to select any pair of frequencies, we should train it to select from maybe 3, 5, 7 fixed sets of pairs of frequencies. This helps with the exploding number of parameters, also make it much easier to train the selector

FINAL THOUGHT:

- I believe this selector might be useful for FFT solver only, if we improve the architecture a little bit it might be useful. Was the chief engineer's selector for FFT something similar or was it completely different?
- While the full 'score' field is not as complex as the full gradient field, it's still quite hefty to train. I was training a 64x64 selector with n_train = 4096 and it never finished (even after 2 days)
- This implementation helps me learn a lot more about this direction, but ultimately it is not the model for FNO. Maybe regular FFT solver, but not FNO